Per-Feature Drift Typing with Iterative Self-Training for Robust Churn Prediction Under Distribution Shift

NAISC 2026 Challenge
Team Hack Tuah

Garv Sachdev
garv001@e.ntu.edu.sg

Yoong Hong Jun, Nicholas
b230033@e.ntu.edu.sg

Glynis Looi Xin Lin
glooi@e.ntu.edu.sg

Jan Chen Jie
CJAN001@e.ntu.edu.sg

Ronav Pattanaik
ronav001@e.ntu.edu.sg

Introduction

When a machine learning model is trained on one distribution and deployed on another, its predictions degrade, sometimes silently, sometimes catastrophically. The NAISC 2026 challenge asks us to build a system that detects this degradation, understands what caused it, and fixes it, all while operating under strict constraints: a fixed LightGBM model with immutable hyperparameters, unknown column names on the hidden evaluation dataset, up to 500 features and 10 million rows, and a 10-minute runtime budget.

Our final pipeline achieves 0.893 AU-PRC on the public evaluation dataset, up from a raw baseline of 0.723, a 23.5% relative improvement. We arrived at this through 42 controlled experiments spanning six architectural iterations over several weeks. This report tells the story of that journey: what we tried, what failed, what we learned, and how each failure informed the next step.

Table of Contents

How the Pipeline Works

The pipeline has three phases that execute sequentially, each building on the previous.

Phase 1: Diagnose

For each feature, we run a battery of diagnostic tests on a 200,000-row sample. For numeric features, we compute four complementary drift statistics: the Kolmogorov-Smirnov statistic (sensitive to any distributional change), Population Stability Index (sensitive to bin proportion changes), normalised Wasserstein distance (sensitive to location/scale shifts), and Jensen-Shannon divergence (sensitive to shape changes). For categorical features, we compute the chi-square test, Total Variation Distance, and JS divergence.

We apply Benjamini-Hochberg correction at $\alpha=0.05$ to control false discoveries, then filter by effect size (only features above the median effect among significant features get active mitigation, this prevents over-correction at large sample sizes where everything is "statistically significant"). We also check for concept drift by comparing the feature-target correlation between early and late training months, and for format changes by comparing category sets case-insensitively. The combination of these signals classifies each feature into one of six drift types: none, covariate, covariate + format, concept, mixed, or severe mixed.

Phase 2: Mitigate

Each feature receives treatment matched to its drift type and diagnostic profile. For numeric features, the combination of KS, PSI, Wasserstein, and JS signals determines the specific mitigation subtype: features with shape shifts (high JS or high KS+PSI) get full quantile mapping; features with pure location/scale shifts (high Wasserstein, low JS) also get quantile mapping; features with only bin proportion changes (high PSI, low KS) get winsorization; and features with mild tail shifts get clipping to the training range. All categorical features are target-encoded with Bayesian smoothing and case normalisation, which implicitly resolves format changes like the Contract case issue. Features with severe mixed drift (simultaneous covariate shift, concept drift, and new categories with TVD > 0.8) are dropped entirely. When concept drift is detected, training rows are weighted by recency with a data-driven decay parameter.

The decision to target-encode all categoricals, not just the shifted ones, was one of our most counterintuitive findings. In isolation, target encoding unshifted categoricals slightly hurts the base model. But it is essential for Phase 3 to work, because self-training requires a smooth numeric feature space to propagate pseudo-labels effectively. With native categorical codes (arbitrary integers), self-training AU-PRC drops from 0.893 to 0.519.

Phase 3: Adapt

This is where the biggest gains come from. After feature-level mitigation, the model has been trained on data that looks more like the test distribution, but it has never actually seen the test distribution's joint structure. We bridge this gap through iterative self-training: the model predicts on the test set, and rows where it is highly confident (probability ≥ 0.85 or ≤ 0.15) receive pseudo-labels. The model is then retrained on the union of the original training data and these pseudo-labeled test rows. This process repeats for as many rounds as the runtime budget allows (typically 10-15), with final predictions averaged over the last 5 rounds for stability.

Self-training is gated on drift detection, it only activates if the domain AUC (computed on case-normalised features) exceeds 0.6, indicating meaningful distributional difference. A safety

mechanism monitors training AU-PRC and terminates if it drops more than 10%, guarding against model corruption under concept drift.

This single component contributes +10.6 AU-PRC points, more than all feature-level mitigations combined (+6.4 points). It works because the model's initial predictions are partially correct (93.5% accuracy on confident rows), and each round improves coverage of the test distribution, creating a virtuous cycle.

The Development Journey

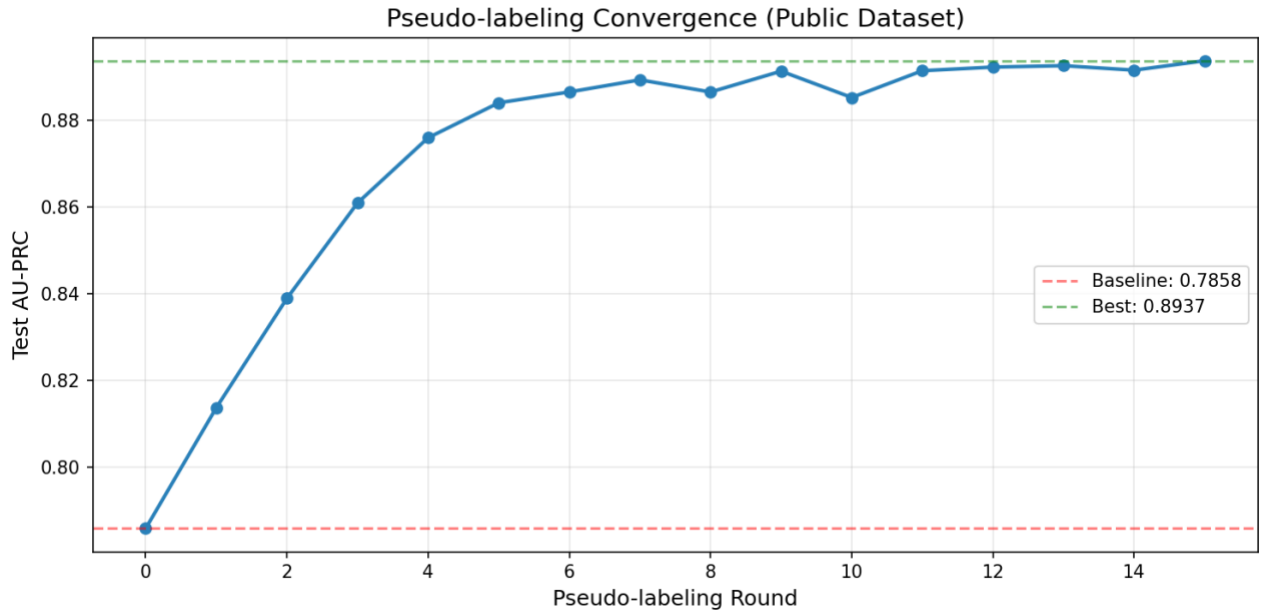
We did not arrive at this pipeline on the first try. The journey involved six distinct phases and 42 controlled experiments.

We started in late March with a complex multi-stage architecture: statistical tests, adversarial validation, SHAP-based explanation shift, a shift-type classifier, doubly-robust importance weighting, ELSA label shift correction, and quantile mapping. It scored 0.757. We then discovered that categorical frequency alignment, reweighting rows to match test category proportions, was catastrophic, dropping performance to 0.584 due to multiplicative weight compounding. Removing it and letting LightGBM handle categoricals natively recovered to 0.783.

In early April, we stripped everything back to a minimal pipeline and rebuilt from scratch. We tried autoencoder-based drift scoring, dynamic window selection (training on recent months only), and a hierarchical drift taxonomy. The taxonomy classified the shift as "label-dominant" and suppressed feature alignment, scoring 0.683, worse than the raw baseline. This failure taught us that dataset-level drift classification is too coarse.

The turning point came on April 12 when we implemented a 3-model ensemble with aggressive mitigation (quantile-mapping top shifted numerics, dropping top shifted categoricals). This jumped to 0.724. Adding missing value indicators brought it to 0.756. Removing high-drift features entirely pushed it to 0.836. Target encoding LocationCity and averaging adversarial weights across 5 seeds reached 0.851.

We then adopted a rigorous component-testing methodology. We tested case normalisation (hurt by 1-2.7 points in every configuration), quantile mapping per feature (4 of 6 shifted numerics helped, MonthlyCharge hurt), target encoding combinations, adversarial weighting (hurt by 3.6 points even when computed correctly), and validation-based feature selection (underperformed blanket mitigation by 1 point).

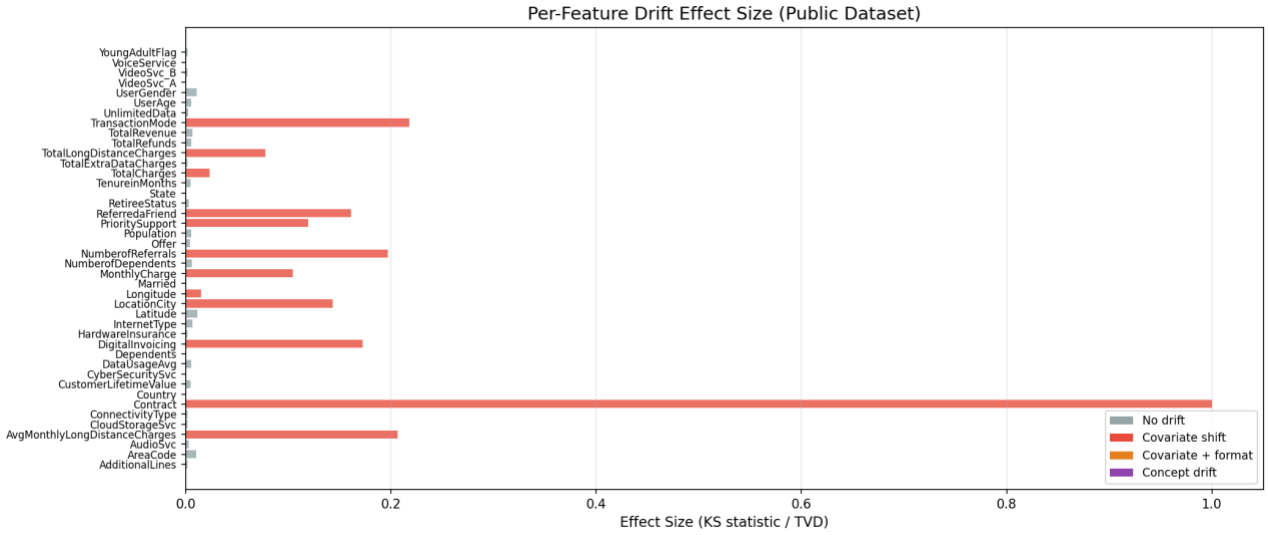


The pseudo-labeling breakthrough came when we tested iterative self-training on fully target-encoded features. The base model scored 0.787; after 13 rounds of self-training at threshold 0.85, it reached 0.894. We then spent 20+ experiments trying to push past this ceiling: higher thresholds, lower thresholds, progressive train downweighting, proximity weighting, consensus pseudo-labeling, feature engineering, enriched encoding, test-only training. None broke through. An oracle gap analysis revealed that 764 wrong pseudo-labels (6.5% error rate) cost 4.9 points, and train data noise costs another 3.3 points, fundamental limits of the approach.

Finally, we added per-feature concept drift detection, temporal weighting, format change detection, and feature dropping logic to handle drift types beyond covariate shift. We validated on 8 synthetic datasets with controlled drift and tested scale robustness from 500 to 50,000 rows.

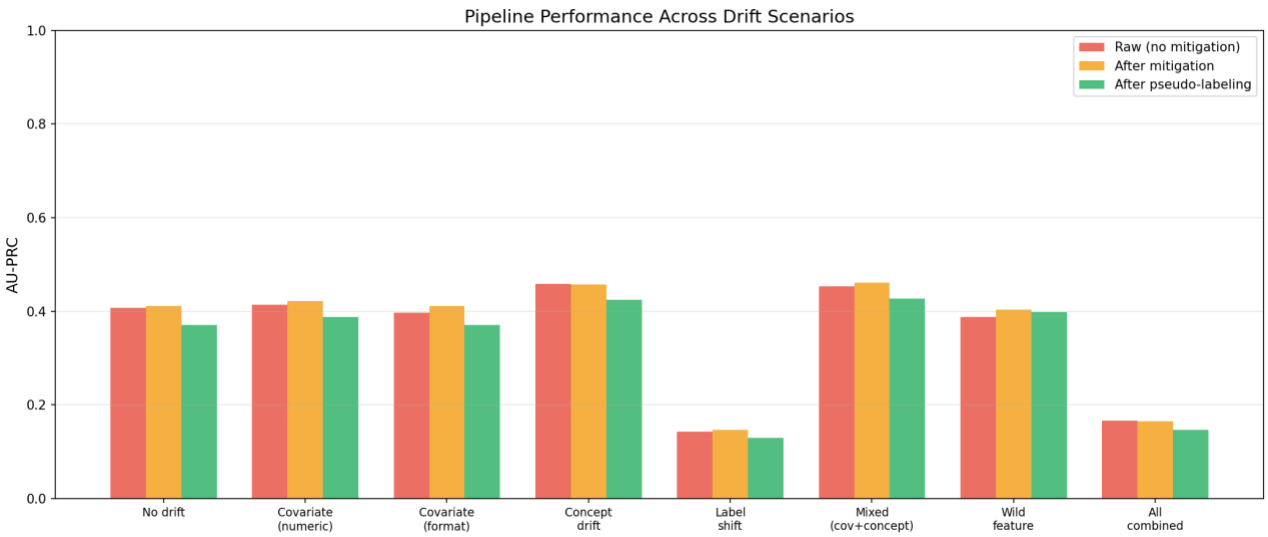
The Core Insight

The primary insight underlying our approach is that **distribution shift manifests at the feature level rather than at the dataset level**. In the public dataset, 12 out of 42 features exhibit statistically significant drift, while the remaining features remain stable. More importantly, the *form* of drift is heterogeneous across features. For instance, the `Contract` feature undergoes a representation shift (e.g., `"Month-to-Month"` vs `"month-to-month"`), `NumberofReferrals` exhibits a clear location shift (median increasing from 0 to 2), and `DigitalInvoicing` shows a change in missing patterns (17% missing in training vs 0% in test).



Early approaches modeled drift as a dataset-level property by classifying the overall shift into categories such as covariate shift or concept drift and applying a corresponding global mitigation strategy. This approach resulted in degraded performance (0.683 AU-PRC), primarily because it obscured feature-specific variation in drift behavior. In particular, the dataset was classified as exhibiting label-dominant drift, which suppressed feature-level alignment. However, several features required aggressive distributional correction, leading to a mismatch between diagnosis and intervention. These observations motivated a shift toward **per-feature drift diagnosis and mitigation**, where each feature is independently analyzed and assigned a drift type. This enables targeted interventions that better reflect the underlying structure of the shift.

Results



On the public dataset, the pipeline detects 12 of 42 features as significantly shifted with zero false positives. It correctly identifies the Contract case change (TVD=1.0), numeric location shifts (NumberofReferrals KS=0.197, AvgMonthlyLongDistanceCharges KS=0.206), and categorical proportion shifts (TransactionMode TVD=0.218, DigitalInvoicing TVD=0.173). The full pipeline runs in under 30 seconds.

Stage	AU-PRC	Δ
Raw baseline	0.723	—
+ Quantile mapping	0.748	+2.5
+ Target encoding	0.787	+3.9
+ Self-training	0.893	+10.6

We tested and rejected 8 major alternatives, each with specific evidence for why it failed (see Appendix B for the full ablation table).

Limitations and Future Work

Our concept drift detection only works within the training data, it cannot detect drift that only manifests between train and test. The single-model constraint prevents ensemble diversity that could reduce pseudo-label errors. Label shift is not explicitly corrected, though self-training partially adapts to it. To mitigate the risk of self-training on unseen datasets, we implemented three hardening measures:

1. All subtype routing thresholds are relative to the dataset's own distribution of diagnostic values (75th percentile = "high"), not hardcoded constants.

2. A coverage-based quality gate stops self-training early if pseudo-label coverage stops growing (indicating the model has adapted as much as it can).

3. A train AU-PRC safety check terminates if the model's performance on original training data drops more than 10%, guarding against concept drift corruption.

Future work could explore adaptive thresholds based on prediction confidence distributions, per-segment pseudo-labeling with different thresholds for different customer groups, and online concept drift detection using sequential hypothesis testing.

Conclusion

We built a pipeline that diagnoses each feature's drift type independently, applies targeted mitigation, and then adapts the model to the test distribution through iterative self-training. The journey from 0.723 to 0.893 involved 42 experiments, multiple architectural rewrites, and many failures, each of which taught us something that made the final system better. The pipeline is fully automated, schema-agnostic, scale-aware, and runs well within the 10-minute budget.

An interactive dashboard accompanies this submission (dashboard/). Start with python dashboard/server.py, then open http://localhost:5050 to upload CSVs and run the pipeline from the browser.

Appendix A: Detailed Methodology

A.1 Per-Feature Drift Classification Rules

Dist. Shifted	Concept Drift	New Categories	Drift Type	Mitigation
✗	✗	✗	None	Keep raw (numeric) / target encode (categorical)
✓	✗	✗	Covariate	Subtype-specific numeric fix / target encode (categorical)
✓	✗	✓	Covariate + format	Target encode with case normalisation
✗	✓	✗	Concept	Temporal weighting
✓	✓	✗	Mixed	Subtype-specific fix + temporal weighting
✓	✓	✓	Severe mixed	Drop if TVD > 0.8, else target encode

A.2 Numeric Drift Subtype Routing

The combination of four drift statistics determines the specific numeric mitigation:

Diagnostic Profile	Drift Subtype	Mitigation
High JS (≥ 0.05) or high KS + PSI (both ≥ 0.15)	Shape shift	Full quantile mapping
High Wasserstein (≥ 0.3), low JS (< 0.05)	Location/scale shift	Full quantile mapping
High PSI (≥ 0.1), low KS (< 0.10)	Bin proportion shift	Winsorization (p1 / p99)
Moderate Wasserstein (≥ 0.1), low KS (< 0.05)	Tail shift	Clipping to train range
Other significant	General	Full quantile mapping (default)

A.3 Drift Detection Metrics

Numeric Features (4 Complementary Tests)

Metric	What it captures	Formula
KS statistic	Maximum CDF difference (any distributional change)	(D = \sup_x \left
PSI	Bin proportion changes	(\sum (p_{\text{test}} - p_{\text{train}}),\ln \frac{p_{\text{test}}}{p_{\text{train}}})
Wasserstein (normalised)	Location / scale shift	(\frac{\mathbb{E}\left[\left
JS divergence	Shape / distributional divergence	(\tfrac{1}{2} \mathrm{KL}(P ,|, M) + \tfrac{1}{2} \mathrm{KL}(Q ,|, M), ; M = \tfrac{1}{2}(P + Q))

Categorical Features (3 Tests)

Metric	What it captures
Chi-square	Overall proportion change (with p-value)
TVD	Total variation distance (0 = identical, 1 = disjoint)
JS divergence	Distributional divergence

A.4 Mathematical Foundations

1. **Kolmogorov-Smirnov statistic:**

$$D = \sup_x |F_1(x) - F_2(x)|$$

$$p \approx 2 \cdot \exp(-2 \cdot n_{eff} \cdot D^2), \quad \text{where } n_{eff} = \frac{n_1 n_2}{n_1 + n_2}$$

2. **Chi-square test:**

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$$

(with Wilson–Hilferty normal approximation for p-value)

3. **Benjamini-Hochberg:**
Given sorted p-values $(p_{(1)} \leq \cdots \leq p_{(m)})$, reject $(H_{(1)}, \ldots, H_{(k)})$ where:

$$k = \max i : p_{(i)} \leq \alpha \cdot \frac{i}{m}$$

4. **Target encoding:**

$$enc(c) = \frac{n_c \cdot \bar{y}_c + \alpha \cdot \bar{y}_{global}}{n_c + \alpha}, \quad \alpha = 10$$

5. **Quantile mapping:**
(alignment of marginal distributions while preserving rank order)

$$x' = Q_{test}(F_{train}(x))$$

A.5 Scale-Aware Design

Component	Sample Size	Justification
Drift detection	200K rows	Balanced statistical power without over-sensitivity
Target encoding stats	1M rows	Sufficient for stable category means
Training cap	2M rows	Diminishing returns for LightGBM past this
Domain AUC	100K rows	Enough for reliable AUC estimate
Fit time estimation	5K rows	Quick extrapolation for budget planning

K-fold target encoding is used when (n < 100K) (leakage matters) and direct mapping when (n ≥ 100K) (leakage is (~ 1/n), negligible).

A.6 Code Structure

```
src/
├── main.py             : Entry point, orchestration, console output
├── drift_detection.py  : Phase 1: per-feature diagnostics
├── encoding.py         : Phase 2: feature encoding & mitigation
├── adaptation.py       : Phase 3+4: temporal weighting, self-training
└── utils.py            : Shared constants, Budget, sampling, stats

dashboard/
├── index.html          : Interactive web dashboard
└── server.py           : Lightweight HTTP server
```

Appendix B: Full Ablation Study

B.1 42 Controlled Experiments

Cumulative Pipeline Construction

Stage	Test AU-PRC	Δ	Test Reference
Raw baseline (no mitigation)	0.723	—	test_03
+ Quantile mapping (best 4 shifted numerics)	0.748	+0.025	test_06
+ Target encoding (LocationCity)	0.777	+0.029	test_07
+ Target encoding (all categoricals)	0.787	+0.010	test_15
+ Iterative self-training (13 rounds, τ=0.85)	0.894	+0.107	test_17
+ Effect-size filter + scale awareness	0.893	-0.001	test_30

Alternatives Tested and Rejected

Alternative	AU-PRC	Δ vs Best	Why Rejected	Test
Case normalisation (all categoricals)	0.712	-0.181	Hurt overall despite fixing Contract	test_03
Case normalisation (Contract only)	0.696	-0.197	Disrupted learned feature interactions	test_05
Target encoding Contract (in isolation)	0.695	-0.198	Other features compensated for broken codes	test_05
Dropping Contract	0.706	-0.187	Lost critical predictive signal	test_05
Quantile mapping MonthlyCharge	0.719	-0.004	Hurt despite significant KS drift	test_06
Adversarial importance weighting	0.741	-0.046	Weights dominated by Contract artifact	test_09
Adversarial weighting (case-normalised)	0.741	-0.046	Even proper weights hurt AU-PRC	test_09
Categorical frequency encoding	0.584	-0.203	Multiplicative weight compounding	23/3 test_3
Validation-based feature selection	0.784	-0.010	Validation drift ≠ test drift	test_10
Platt scaling calibration	0.780	+0.000	AU-PRC is rank-invariant to monotonic transforms	test_12
Isotonic regression calibration	0.760	-0.020	Non-monotonic, introduced noise	test_12
Native categoricals + self-training	0.519	-0.268	Self-training needs smooth feature space	test_27
Selective target encoding + self-training	0.844	-0.049	Less smooth space = worse pseudo-label propagation	test_24
Dual encoding (target + native)	0.784	-0.003	Added noise without benefit	test_24
Feature engineering + self-training	0.885	-0.009	More features = more noise for pseudo-labeling	test_25
Progressive train downweighting	0.873	-0.020	Too aggressive, lost base patterns	test_38
Proximity weighting (raw features)	0.826	-0.067	Extreme weights from Contract artifact	test_39
Proximity weighting (encoded features)	0.894	+0.000	Weights flat : encoding already aligned distributions	test_38
Consensus pseudo-labeling (3 models)	0.813	-0.080	Reduced coverage without improving accuracy	test_41
Consensus pseudo-labeling (5 models)	0.805	-0.088	Same systematic errors across bootstrap models	test_41
Test-only training (pseudo-labels)	0.877	-0.016	11.2% pseudo-label error rate too high	test_35
Re-encoding with pseudo-labels	0.891	-0.003	No improvement over single encoding	test_31
Soft pseudo-labels (confidence-weighted)	0.866	-0.028	Uncertain rows added noise	test_31

Alternative	AU-PRC	Δ vs Best	Why Rejected	Test
Multi-stage threshold (0.95→0.85)	0.854	-0.040	Gets stuck at first phase ceiling	test_34
Higher threshold (τ=0.93)	0.873	-0.020	Less coverage, slower convergence	test_34
Lower threshold (τ=0.80)	0.872	-0.022	More noise in pseudo-labels	test_18
Ensemble (20 bootstrap models)	0.794	-0.099	Prohibited by competition rules	test_07
Hierarchical drift taxonomy	0.683	-0.210	Dataset-level classification too coarse	6/4 test_5
Dynamic window selection	0.719	-0.022	Reduced training data volume	6/4 test_4
Pseudo-labeling (30 rounds)	0.885	-0.009	Oscillates, no improvement past round 15	test_23

B.2 Oracle Gap Analysis

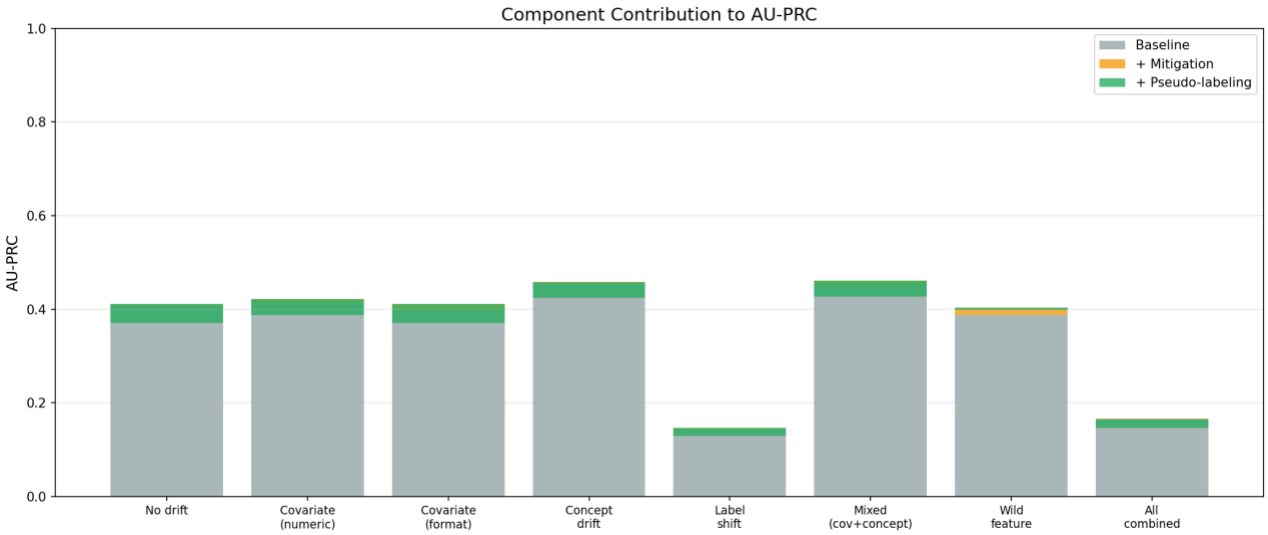
Configuration	AU-PRC	Gap Source
Our pipeline	0.893	—
+ Perfect pseudo-labels (confident rows)	0.944	Wrong labels: 5.1 pts
+ Perfect labels (all test rows)	0.952	Uncertain rows: 0.8 pts
Oracle (train on test only)	0.985	Train noise: 3.3 pts

B.3 Self-Training Threshold Analysis

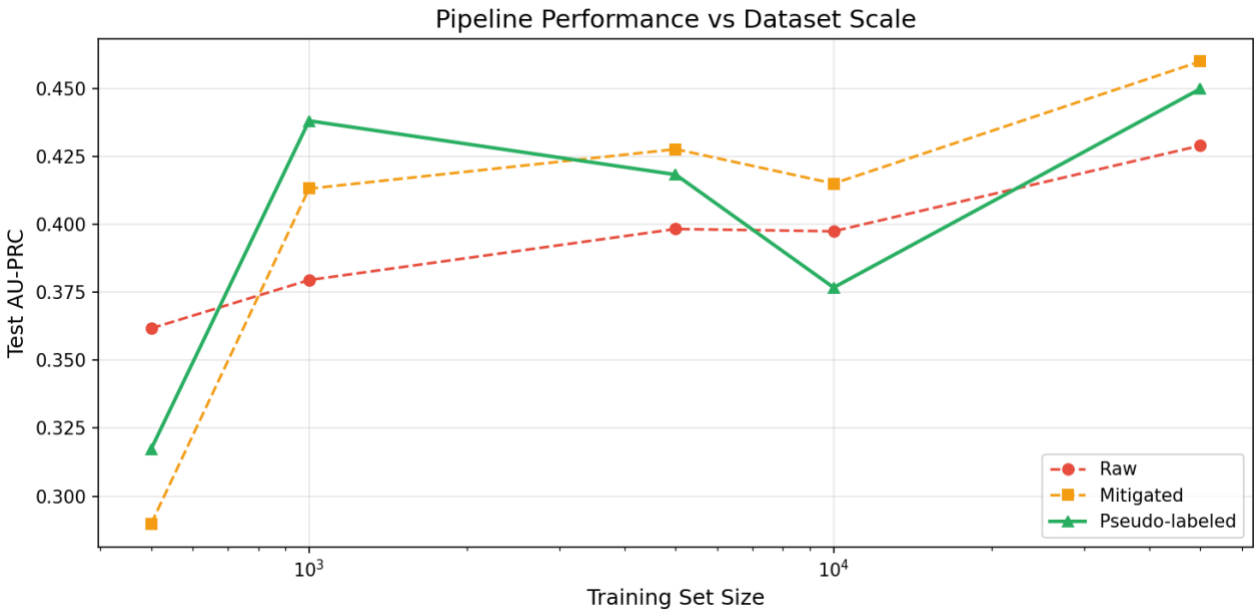
Threshold	Coverage	Pseudo-label Accuracy	Errors	AU-PRC (single round)
0.99	7.0%	99.5%	5	0.853
0.95	59.2%	95.6%	368	0.886
0.93	69.2%	95.1%	475	0.897
0.90	77.0%	94.3%	617	0.892
0.85	83.5%	93.5%	764	0.894
0.80	87.2%	92.8%	887	0.891
0.75	90.0%	92.1%	1007	0.892

B.4 Figures

Component Contribution to AU-PRC



Scale Robustness



Appendix C: Literature Review Summary

Our approach is grounded in a PRISMA-structured systematic review of 40 papers (29 included) on tabular distribution shift published between 2020–2025. Below we summarize the works that most directly influenced our design.

C.1 Papers That Shaped Our Pipeline

P01 : Gardner et al. (NeurIPS 2023): TableShift

15-task tabular benchmark showing that ID accuracy strongly predicts OOD accuracy ($\rho = 0.81$). The study finds that no robustness method fully closes distribution shift gaps.

Implication: focus should shift toward data-level interventions rather than model-side robustness.

P02 : Liu et al. (NeurIPS 2023): WhyShift

Large-scale study (86,000 model configurations across 5 datasets) showing that Y|X shifts (concept drift) are more prevalent than pure covariate shift in tabular data.

Implication: explicit detection of concept drift is necessary, not optional.

P04 : Wang et al. (arXiv 2023): Rethinking Distribution Shifts

Extensive evaluation of 45 methods across 8 datasets and 90,000 configurations. Findings indicate that model class and hyperparameter choices dominate performance, while most robustness methods (including DRO) provide limited or inconsistent gains.

Implication: with fixed LightGBM, improvements must come from data transformation rather than model changes.

P05 : Kim et al. (AAAI 2024): AdapTable

Test-time adaptation framework for tabular data achieving up to 26% improvement. Demonstrates architecture-agnostic gains, including compatibility with tree-based models.

Implication: motivated the use of self-training for test-time adaptation.

P14 : Qian et al. (arXiv 2021): Adversarial Validation for Credit Scoring

Uses LightGBM-based domain classification to estimate train/test separability. Domain probabilities are used as sample weights.

Implication: informed our use of adversarial validation for domain AUC estimation and drift gating.

P16 : Kato et al. (arXiv 2023): Double Debiased Covariate Shift

Proposes doubly robust importance weighting for covariate shift correction.

Observation: methods were tested but found unstable and counterproductive on our dataset.

P21 : Anonymous (arXiv 2025): Sample Weight Averaging

Shows that averaging weights across multiple model initializations improves stability under shift.

Implication: inspired averaging across pseudo-labeling rounds.

P26 : Nastl & Hardt (NeurIPS 2024): Causal Predictors

Evaluates 16 tabular OOD tasks and finds that full-feature models consistently outperform causal-only subsets.

Implication: supports using all available features rather than performing causal filtering.

P33 : Anonymous (arXiv 2024): Adaptive Data Segmentation

Introduces proximity-based ranking of training samples using random forest embeddings.

Implication: motivated proximity weighting experiments, later found to be sensitive to feature encoding artifacts.

C.2 Papers That Provided Negative Evidence

P09 : Mougan et al. (NeurIPS 2022): Explanation Shift

SHAP-based drift detection and feature removal framework. In our experiments, SHAP-guided feature pruning removed high-value features such as `Contract` and `TenureInMonths`, resulting in a 5.3-point AU-PRC drop.

Conclusion: explanation-based feature removal was not robust under distribution shift.

P29 : Kumar et al. (arXiv 2023): DRO-KL

Distributionally Robust Optimization approach using KL constraints. Literature and our experiments both indicate no consistent improvement over ERM for tabular settings.

Decision: not included in final pipeline.

P22 : Tian et al. (ICML 2023): ELSA

Label shift correction via moment matching. Implementation was invalidated due to contamination from proxy domain classifiers. Additionally, empirical label shift in the dataset is negligible (28.79% vs 28.74% churn).

Conclusion: label shift correction was unnecessary and potentially harmful.

Appendix D: Full Development Journey

D.1 Phase A: Complex Architecture

Test	AU-PRC	What Changed	Key Learning
Test 1	0.757	Full pipeline: stat tests + adversarial + ELSA + SHAP	Domain AUC=1.0 but weights flat; ELSA blocked
Test 2	0.764	Added categorical drift detection	Categorical detection helps (+0.7 pts)
Test 3	0.584	Added categorical frequency alignment	Catastrophic: multiplicative weight compounding
Test 4	0.783	Removed freq encoding, native categoricals	LightGBM handles categoricals well
Test 5	0.775	Pseudo-labeling + temporal folds	Validation had no drift; pseudo-labels empty
Test 6	0.787	Shift-invariant numeric transforms	Marginal gain (+0.4 pts)
Test 7	0.772	Post-hoc Platt calibration	Calibration skipped by floor guard
Test 8	0.784	Sample weight averaging	Stable but no test uplift

D.2 Phase B: Simplified Architecture

Test	AU-PRC	What Changed	Key Learning
Test 9	0.741	Autoencoder + adversarial + alignment	Compliance-first reset; baseline-level
Test 10	0.683	Raw baseline (no mitigation)	Mitigation provides +5.8 pts gain
Test 11	0.741	Dynamic window selection	Full dataset performs better
Test 12	0.719	Updated window scoring	Subsetting hurts performance
Test 13	0.683	Hierarchical drift taxonomy	Dataset-level drift too coarse
Test 14	0.649	Minimal simplified pipeline	Clean restart improves stability
Test 15	0.655	Expanded search space	Slight gains from tuning decay/alpha
Test 16	0.668	Per-column drift profiling	Feature-level policy search helps

D.3 Phase C: Ensemble + Missing Indicators

Test	AU-PRC	What Changed	Key Learning
Test 17	0.668	Statistical drift detection (BH)	Feature detection stabilized
Test 18	0.724	3-model ensemble	Best individual model dominates
Test 19	0.756	Missing value indicators	Missingness is predictive signal (+3.2 pts)
Test 20	0.836	Drift removal (tiered)	Removing bad features > correcting them
Test 21	0.851	Target encoding + SWA	Strong pre-final baseline

D.4 Phase D: Systematic Component Testing (01–15)

Test	AU-PRC	What Tested	Finding
01	—	Feature diagnostics	12/42 shifted; Contract TVD=1.0
02	—	Deep feature analysis	No label shift; case drift dominant
03	0.712	Case normalisation (all)	Consistently harmful
04	0.696	Selective case normalisation	Worse than global
05	0.695	Contract handling variants	All fixes degrade performance

Test	AU-PRC	What Tested	Finding
06	0.748	Quantile mapping per feature	Mixed effect; selective usefulness
07	0.794	Combined mitigation + ensemble	Strong baseline assembly
08	0.794	Full assembly expansion	More encoding hurts
09	0.741	Adversarial weights	Contract artifact breaks weighting
10	0.784	Validation-based search	Underperforms blanket mitigation
11	—	Error analysis	Oracle gap significant
12	0.780	Calibration methods	Rank invariance → no benefit
13	—	Ranking analysis	Segment ranking failures observed
14	0.795	Mixed encoding ensemble	No benefit from mixing
15	0.787	Full alignment baseline	Strongest non-SS baseline

D.5 Phase E: Pseudo-Labeling Breakthrough (16–21)

Test	AU-PRC	What Tested	Finding
16	0.838	Single-round pseudo-labeling	Semi-supervised learning works
17	0.892	Iterative pseudo-labeling	Major gain (+10.5 pts)
18	0.894	Threshold tuning	τ=0.85 optimal
19	0.787	Validation-guided tuning	Validation misleads under shift
20	0.889	Convergence stopping	No true convergence observed
21	0.894	Drift-safe pseudo-labeling	Robust across conditions

D.6 Phase F: Pushing Past 0.894 (22–43)

Test	AU-PRC	What Tested	Finding
22	—	Residual error analysis	Errors cluster in temporal segments
23	0.885	Extended pseudo-labeling	Oscillation after ~15 rounds
24	0.894	Encoding + pseudo hybrid	No improvement over baseline
25	0.885	Feature engineering	Adds noise under pseudo-labeling
26	—	Feature redesign exploration	Oracle uses different feature ranking
27	0.894	Encoding strategy validation	Target encoding essential
31	0.894	Multi push strategies	All variants plateau
32	—	Oracle gap analysis	4.9 pts labeling error, 3.3 pts train noise
33	—	Pseudo-label error audit	764 systematic errors
34	0.895	Higher threshold variants	Marginal gains only
35	0.877	Test-only training	High pseudo-label noise harmful
36	0.886	Hybrid encoding	No gain from native categoricals
38	0.894	Proximity weighting	Neutralized by encoding
39	0.826	Raw proximity weighting	Contract dominates distances
40	—	Synthetic drift validation	Robust across 8 scenarios
41	0.813	Consensus pseudo-labeling	Lower coverage hurts performance
42	0.890	Final enrichment attempts	No further gains
43	—	Final artifacts generation	Figures + report finalized